

Technical Design Document

Nimble Sentences With Reinforcement Learning

Tyler Dinh, Lauren Gayle, Anling Zhu, Tenzin Dukdak,
Billy Zhao, Noah Wolk, Mark Seidel, Horlasy Demakpor

Software Team

{dinh.t, gayle.l, anling, dtenzin, billy.z,
n.j.wolk, m.m.seidel, horlasy.d}@wustl.edu

Elizabeth Joseph, Alexis Luna, Yuxuan Shen,
Kyle Okui, Gabe Minton

Integration Team

{elizabeth.j, alexis.l, notsure, k.okui, g.m.minton}@wustl.edu

February 10, 2026

Contents

1	Overview	3
2	Problem	4
2.1	The Core Problem	4
2.2	The Core Goal	4
3	Tenets	5
4	Requirements	6
4.1	Out of Scope	6
4.2	Success Criteria	6
5	Architecture	7
5.1	High-Level Overview (HLD)	7
5.2	API Design	7
5.3	Data Storage and Model	7
5.4	Application / Component Level Design (LLD)	7
6	Dependencies	8
6.1	Design Alternatives Considered	8
7	Cost Analysis	9
8	Failures and Risks	10
9	Non-Functional Requirements	11
9.1	Scalability	11
9.2	Latency and Availability	11
9.3	Maintainability	11
9.4	Security	11
10	Testing and Observability	12
10.1	Testing	12
10.2	Metrics and Alarms	12
11	Future Improvements	13
12	FAQs	14
A	Appendix A: Subtitle (Optional)	15
B	Glossary	16
C	References	17

1 Overview

The purpose of this document is to present the technical design of a system that enables a robotic arm to learn and perform handwriting tasks for basic sentences using reinforcement learning (RL).

This document is intended for a technical audience with a background in computer science, robotics, or machine learning. Non-technical readers may find high-level summaries useful but should refer to supplementary documentation at the end of this document for background on the underlying mechanisms employed.

2 Problem

Current robot arms used for writing tasks operate with **static, pre-programmed motions**. Each specific writing task—whether it’s writing a particular sentence, drawing a shape, or creating a signature—requires its own custom motion program. This approach has several critical limitations:

2.1 The Core Problem

When a robot needs to write something new, engineers must:

1. Manually program the exact trajectory for each letter or word.
2. Hard-code the sequence of movements.
3. Test, refine, and repeat this entire process for every new writing task.

A **dynamic robot arm** that can generalize its writing capabilities would:

- **Scale efficiently:** Write any arbitrary text without reprogramming.
- **Adapt to variations:** Handle different sizes and styles of text.
- **Learn from examples:** Improve through demonstration rather than explicit programming.

2.2 The Core Goal

The goal is to develop a robot writing system that can:

- Accept arbitrary text input (like “Hello World” or even more arbitrary, like “Write me a haiku.”)
- Dynamically generate appropriate writing motions.
- Execute smooth, legible handwriting without pre-programmed trajectories for each specific string.

3 Tenets

TODO! List the key principles or beliefs guiding the design decisions. Tenets help align teams and establish common ground on critical design questions. talk to team about this

4 Requirements

TODO! Describe all requirements imposed by the problem. Write from the end-user's perspective. Include user stories or use cases.

Example Use Cases

- As a x, I want to y.
- As a x, I want y.

4.1 Out of Scope

TODO! List what is explicitly out of scope to avoid misunderstandings.

4.2 Success Criteria

TODO! Describe how success will be measured once the solution is in production. Include measurable metrics such as performance, scalability, or cost efficiency.

Example Metrics

- Time-to-market reduced by 90%.
- Supports 10,000 users with 100ms P90 latency.
- It can write sentences in English legibly with X% accuracy.

5 Architecture

Describe the architecture of the proposed solution using text, diagrams, and bullet points. Explain why this design was chosen.

25-26 Architecture

VLA: NORA 1.5 Data Collection Framework: LeRobot Arm: WidowX AI Trossen Base CV: OpenCV + Logitech Additional VLM/LLM for Data Preprocessing: Local Qwen3-4B instance, Programming Languages: Mainly Python, JSON for data. Compute: RoboServer

5.1 High-Level Overview (HLD)

List all logical system components:

- comp1
- comp2
- comp3
- comp4
- comp5

Include diagrams as needed:

5.2 API Design

List all APIs used for interaction between users or services. Include HTTP methods, payloads, versions, and example requests/responses. Discuss future evolution of the APIs.

5.3 Data Storage and Model

Describe the data model and database choice. Estimate data volume, forecast growth, and justify scalability. Discuss data pipelines, ingestion, and pre-processing if applicable.

5.4 Application / Component Level Design (LLD)

Provide detailed design of each system component, including data flow and control flow diagrams.

USE DRAW.IO for this (can install on your computer)

6 Dependencies

List external and internal dependencies. Document assumptions and risks associated with these dependencies.

6.1 Design Alternatives Considered

Discuss alternative designs evaluated. Provide a comparison table highlighting trade-offs.

Option	Pros	Cons
Option A	Scalable, cost-efficient	Complex to maintain
Option B	Easy to implement	Limited scalability

7 Cost Analysis

Estimate infrastructure and operational costs. Plan for future growth and scalability.

8 Failures and Risks

Discuss potential system failures such as:

- Dependency failures
- Traffic overflow
- Performance degradation
- Logic bugs

Identify risks such as external dependencies or resource limitations. Include potential mitigation strategies.

9 Non-Functional Requirements

Cover scalability, availability, maintainability, reliability, latency, and security.

9.1 Scalability

Expected users, transactions, and data volume.

9.2 Latency and Availability

Define SLAs (e.g., P99, P50 latency targets).

9.3 Maintainability

Explain maintenance expectations and ownership.

9.4 Security

Describe security measures and required compliance levels.

10 Testing and Observability

Outline strategies for testing, monitoring, and alerting.

10.1 Testing

Explain testing approach: unit, integration, A/B, or stress tests.

10.2 Metrics and Alarms

List key performance metrics, dashboards, and alert configurations.

11 Future Improvements

List planned features or improvements for future releases.

12 FAQs

Include frequently asked questions and their answers.
question? answer here

A Appendix A: Subtitle (Optional)

Include detailed technical analysis or supporting data.

B Glossary

ex1 example definition

C References

- Service 1: <https://example.com>